ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



HỌC PHẦN: AN TOÀN PHẦN MỀM

ĐỀ TÀI: FUZZING WHITE BOX TESTING CHO QUẢN LÝ DÒNG HỌ

Giảng viên: ThS. Vũ Quang Dũng

Nhóm 5:

3. Triệu Tiến Quỳnh (23010648 - K17 - KTPM(EL_2))

Lớp học phần: CSE703093-1-1-26(N02)

Hà Nội, tháng 10 năm 2026

DANH SÁCH PHÂN CÔNG

STT	Mã sinh viên	Họ và tên	Công việc
3	23010648	Triệu Tiến Quỳnh	Phần 3.5, 3.6 (Phân tích tĩnh ASan/UBSan, phép đo độ phủ) Phần 5.1, 5.2 (Ý nghĩa an toàn phần mềm, giới hạn soundness / completeness). Phần 4.4, 4.5 (Kiểm thử biên, đo sizeof vùng nhớ) Phần 4.8, 4.9 (Giới hạn thuật toán nhận dạng path, đối chiếu XLSX) Phụ lục A

3. Phương pháp thực hiện

3.5. Phân tích tĩnh và thực thi bằng sanitizer

Quá trình kiểm tra đối chiếu ba lớp: điều kiện thực sự chạy, thao tác bộ nhớ và cơ chế sinh kết quả. README và bốn sheet XLSX được dùng làm tài liệu tham khảo, nhưng kết luận dựa vào code cùng kết quả chạy lại. Quy mô mã nguồn được đếm theo dòng vật lý; kích thước bộ nhớ được đo bằng sizeof trên môi trường thực nghiệm.

AddressSanitizer được chọn để xác nhận thao tác bộ nhớ không hợp lệ trong các đường đi đã thực thi. UndefinedBehaviorSanitizer được bật cùng để bổ sung kiểm tra các hành vi không xác định được công cụ hỗ trợ [3], [4].

Lệnh biên dịch:

```
clang -std=c11 -Wall -Wextra -g -O0 \
  -fsanitize=address,undefined -fno-omit-frame-pointer \
  source/clan_genealogy_fuzzer.c -o results/baseline
```

Môi trường kiểm tra ngày 23/10/2026 là macOS 15.6.1, ARM64, Apple Clang 17.0.0. -O0 giảm việc tối ưu bỏ thao tác phục vụ kiểm thử; -g và -fno-omit-frame-pointer hỗ trợ lần theo lỗi. Khi chạy, ASAN_OPTIONS=detect_leaks=0 được sử dụng nên không đánh giá memory leak. Mỗi tiến trình có giới hạn 45 giây; ca lỗi độ dài âm bị sanitizer chấm dứt trước giới hạn này.

3.6. Chiến dịch và các phép đo độ phủ

Chiến dịch gồm hai seed, sáu biến thể BFS_QUEUE và ba biến thể DFS_STACK. Các biến thể được gán cố định trong code rồi chạy tuần tự. Không có bộ sinh ngẫu nhiên, giải ràng buộc hay cơ chế chọn đầu vào mới theo phản hồi độ phủ.

Hai hàm duyệt đồ thị vẫn có hoạt động cụ thể: traverse_depth_dfs đi theo chuỗi cha bằng vòng lặp; traverse_children_bfs dùng hàng đợi, xét bốn nút đầu. Các tên BFS/DFS của mảng ca kiểm thử mô tả cách tổ chức các tình huống, chưa chứng minh có thuật toán tự động khám phá không gian trạng thái. Báo cáo vì vậy xem phiên bản này là kiểm thử định sẵn dựa trên cấu trúc mã nguồn và trạng thái, chưa phải một chiến dịch coverage-guided fuzzing hoặc symbolic white-box fuzzing hoàn chỉnh.

State Coverage = số phần tử được đánh dấu trong g_state_visited / 16. Predicate Coverage được tính tương tự với g_predicate_visited. enter_state đánh dấu hai loại nhãn cùng lúc; không có bộ đánh giá độc lập tính lại biểu thức mệnh đề. Path Coverage = số phần tử đã đánh dấu trong g_path_visited / 10. Hàm record_execution_path hiện chỉ so sánh trạng thái cuối với terminal_state của từng path.

Để kiểm tra ý nghĩa của số đo path, báo cáo bổ sung hai việc: so sánh toàn bộ trace quan sát trong baseline với chuỗi khai báo và kiểm thử riêng bộ nhận dạng bằng một trace không khớp nhưng có cùng trạng thái cuối. Hai việc này kiểm tra các khía cạnh khác nhau: tính đúng của dữ liệu lần chạy hiện tại và khả năng phân biệt của bộ đo.

4. Kết quả và phân tích

4.4. Kiểm thử biên và sai lệch mệnh đề

Thử nghiệm	Quan sát trên bản cập nhật	Ý nghĩa
Audit = -1	ASan báo lỗi; tiến trình bị chấm dứt	C-01 vẫn tồn tại
Audit = 0 hoặc 24	S15, PATH_01, mã thoát 0	Không có chẩn đoán ở hai biên được kiểm tra
Audit = 25	S14, PATH_07, mã thoát 0	Cảnh báo mô hình; không có đọc ngoài biên
Fund = 16384	Cần 65536 byte, thu hẹp thành 0; S12 không được ghi	Bộ phát hiện bỏ sót biên đầu tiên mất dữ liệu
Fund = 16385	Cần 65540 byte, thu hẹp thành 4; S12	Được phát hiện theo điều kiện đã cài
Fund = 32768	Cần 131072 byte, thu hẹp thành 0; S12	Một giá trị lớn hơn được phát hiện
Tiểu sử 100% dong gop	S03 rồi PATH_01	P03 không thực sự yêu cầu không có mọi dấu %

Phối ngẫu = 14; ô 14 là NULL	S10 rồi PATH_01	Không kiểm tra tổng quát con trỏ phối ngẫu như mô tả P10
Vòng đời = 3	S05 rồi S07, PATH_09	S07 được gán cả khi giá trị khác 2

Bảng 4.4. Bảng kiểm thử bổ sung. Ngoài audit bằng -1, các tiến trình trên kết thúc với mã 0. PATH_01 ở một ca bất thường không chứng minh các điều kiện nghiệp vụ đã đúng.

Phép tính kích thước ở dòng 356–357 còn điều kiện:

```
uint16_t alloc_bytes =
    (uint16_t)(tc->step4_fund_count * sizeof(uint32_t));
if (tc->step4_fund_count > 16384 &&
    alloc_bytes < tc->step4_fund_count) {
    enter_state(S12_ERR_FUND_INT_OVERFLOW, 12, tc);
    goto finalize_path;
}
```

Ở count 16384, kết quả toán học 65536 bị thu hẹp thành 0, nhưng điều kiện > 16384 sai. Đây là lỗi của bộ phát hiện (C-02). So sánh số byte sau thu hẹp với số bản ghi cũng không trực tiếp kiểm tra kích thước có được bảo toàn. CWE-197 mô tả việc mất dữ liệu khi đổi sang kiểu nhỏ hơn [7]. Phép đổi sang kiểu nguyên không dấu nhỏ hơn không mặc nhiên là undefined behavior, nên không có cảnh báo UBSan ở đây không chứng minh phép tính đúng mục đích.

Các kiểm thử tiểu sử, phối ngẫu và vòng đời còn chứng minh biểu thức trong STATE_TABLE chưa đồng nhất với điều kiện chạy. Ví dụ, P10 ghi con trỏ phối ngẫu khác NULL, nhưng code chỉ kiểm tra ô 15; P07 ghi thao tác bằng 2, nhưng nhánh thực chấp nhận mọi số dương khác 1. Việc đánh dấu nhãn mệnh đề không thay thế được kiểm tra biểu thức.

4.5. Kích thước vùng nhớ trên môi trường thực nghiệm

```
sizeof(ClanMember)=72 sizeof(void*)=8 sizeof(int)=4
```

Cấu trúc mới đã bỏ hai trường liên kết so với bản trước; phép đo hiện tại là 72 byte trên ARM64, không dùng lại con số 80 byte của lần phân tích cũ. Mô tả “64 bytes” trong STATE_TABLE,

README và dữ liệu tham khảo vẫn chưa khớp với phép đo này. Chênh lệch xuất phát từ các trường dữ liệu, con trỏ và phần đệm căn chỉnh; cần đo lại khi đổi ABI.

g_member_table[16] chiếm 128 byte với con trỏ 8 byte. Hàng đợi BFS có 64 số nguyên, chiếm 256 byte với sizeof(int) = 4. Mô tả hàng đợi [64] trong bảng trạng thái mới đã phù hợp với khai báo. Các bộ đệm cục bộ gồm tên 16 byte, tiểu sử 64 byte, nguồn audit 24 byte và đích audit 64 byte. Dòng mô tả "Heap Fund Buffer" chỉ là mô phỏng, vì code không có cấp phát vùng quỹ trong bước 4.

4.8. Giới hạn của bộ nhận dạng path

Mã phát hiện C-04; mức bằng chứng: phân tích tĩnh, kiểm thử đơn vị bộ đo và đối chiếu trace baseline.

Mặc dù chú thích hàm nói đối chiếu chuỗi trạng thái, phần quyết định trong record_execution_path chỉ sử dụng trạng thái cuối:

```
SystemStateID terminal =
  g_current_trace[g_current_trace_len - 1];
    for (int p = 0; p < TOTAL_PATHS; p++) {
        if (PATH_TABLE[p].terminal_state == terminal) {
       g_path_visited[p] = 1;
       /* Ghi nhan path va dung tim kiem. */
       break;
    }
 }
```

Đoạn trên rút gọn phần log và bộ đếm, giữ nguyên tiêu chí so khớp. state_sequence trong bảng path là chuỗi dùng để mô tả/in báo cáo, không được so sánh với trace để quyết định path đã phủ.

Kiểm thử đơn vị dựng trace S00 → S15, ngắn hơn PATH_01, rồi gọi bộ nhận dạng. Kết quả:

```
[PATH COVERAGE TRACE] S00 -> S15
Matched: PATH_01 (Full_Normal_Commit_Path)
PATH_01_visited=1
```

Kết quả này chứng minh bộ nhận dạng không kiểm tra toàn bộ chuỗi. Nó không chứng minh trace rút gọn là một đường đi tự nhiên có thể đạt từ module hiện tại. Với cây trạng thái đang có, mỗi path có trạng thái cuối khác nhau và 11 trace baseline thực tế đều khớp chuỗi khai báo. Vì vậy, số 10/10 của lần chạy hiện tại có thêm bằng chứng từ việc đối chiếu trace, nhưng thuật toán đo vẫn cần được sửa nếu muốn xác thực đường đi đầy đủ hoặc mở rộng mô hình.

Độ phủ còn bỏ qua số lần lặp DFS/BFS, kết quả kiểm tra g_member_table[slot] == NULL, các giá trị khác nhau của từng miền số và các chuỗi cùng đi vào một nhãn. Không nên diễn giải 100% path là đã kiểm thử mọi đường điều khiển của C.

4.9. Đối chiếu README và bảng XLSX cập nhật

README hiện hướng dẫn biên dịch GCC và chạy trực tiếp, mô tả đúng quy mô một module, 16 trạng thái, 16 nhãn mệnh đề, 10 path và 11 ca. Hướng dẫn libFuzzer cùng mô tả 50.000 lượt của bản cũ đã được bỏ, nên không được tiếp tục ghi trong báo cáo như lỗi của README hiện tại.

Bảng XLSX mới có bốn sheet: trạng thái/mệnh đề/vùng nhớ, đặc tả 10 path, 11 ca kiểm thử và log. Cấu trúc và các tổng số phù hợp với phiên bản C mới. Tuy nhiên, các nhãn “CRASH”, “HANG” và kích thước heap 64 byte trong tài liệu vẫn phải được đọc theo giới hạn đã phân tích, không thay thế bằng chứng sanitizer và phép đo sizeof.

Tệp C chưa có entry point LLVMFuzzerTestOneInput; báo cáo không chạy hoặc tuyên bố có chiến dịch libFuzzer. Việc phát triển harness theo giao diện chuẩn [5] chỉ được nêu như hướng cải thiện. Log gốc được gửi ra stdout, vì vậy bằng chứng của lần chạy này nằm trong baseline.stdout.txt, không phải tệp do chương trình tự mở.

5. Thảo luận và hạn chế

5.1. Ý nghĩa của kết quả đối với an toàn phần mềm

Thực nghiệm cho thấy ba lớp đánh giá cần được tách biệt: đạt nhãn trạng thái/path trong mô hình, thực thi hành vi nguy hiểm và tác động lên hệ thống triển khai. Bộ ca gốc làm tốt việc đi qua các nhãn đã được thiết kế. Tuy nhiên, nhiều nhãn chỉ biểu diễn tình huống giả định; chưa có thao tác bộ nhớ tương ứng để tạo lỗi thực. Vì vậy, bộ đếm phản ánh hoạt động của mô hình hơn là số lượng lỗ hổng được công cụ xác nhận.

Trường hợp độ dài âm minh họa lợi ích của kết hợp đọc code và kiểm thử động. Phân tích điều kiện cho thấy chỉ có cận trên; ca kiểm thử -1 biến nhận xét đó thành bằng chứng thực thi. Ngược lại, chỉ đọc log mô phỏng sẽ bỏ qua lỗi này vì cả S14 và S15 đã được đánh dấu sau bộ ca ban đầu.

Đối với ứng dụng web, lỗi bộ nhớ của C không thay thế được việc kiểm tra quyền truy cập dữ liệu. Rủi ro quan trọng có thể xuất hiện tại ranh giới nghiệp vụ, chẳng hạn quyền truy cập bản ghi của dòng họ khác, dù không liên quan đến tràn bộ đệm.

5.2. Độ phủ và giới hạn soundness, completeness

Tỷ lệ 100% trong báo cáo là độ phủ 16 nhãn trạng thái, 16 nhãn mệnh đề và 10 path đã khai báo. Nó không phải độ phủ toàn bộ câu lệnh, nhánh, đường đi, miền số nguyên hay API của ứng dụng. Mỗi trạng thái có thể bao gồm nhiều đầu vào với hành vi khác nhau: nhánh còn lại sau kiểm tra claim_len > 24 bao gồm cả số âm và các giá trị không âm hợp lệ.

Nếu hiểu nhãn “CRASH” là kết luận có crash thực, cơ chế hiện tại tạo ra nhiều cảnh báo không tương ứng với sự cố đã quan sát. Nếu hiểu việc đạt 100% là đã tìm hết lỗi, trường hợp -1 là phản ví dụ cho kết luận đó. Đây là liên hệ thực tế với yêu cầu xem xét tính đúng và tính đầy đủ của công cụ phân tích, nhưng báo cáo không tuyên bố đã chứng minh hình thức soundness hoặc completeness của toàn chương trình. Việc đạt đủ 10 lá giúp kiểm tra cấu trúc mô hình hiện tại, nhưng không thay thế kiểm tra các đường đi bị bộ đo gộp lại.

Phụ lục

Phụ lục A. Tổ chức mã nguồn và bằng chứng

```
BTL_AnToanPhanMem_CapNhat/
  Bao_cao_An_toan_phan_mem_cap_nhat.docx
  README.md
  source/
    clan_genealogy_fuzzer.c
    README_goc.md
    quanly-dongho/
  tools/
    probe.c
    reproduce.py
    validate_traces.py
    check_schema.cjs
  results/
    baseline.stdout.txt
    baseline.stderr.txt
    audit_-1.stderr.txt
    path_mismatch.stdout.txt
    trace_validation.json
    schema_unit.txt
    manifest.json
    source_inventory.json
    workbook_reference.json
```

Mã nguồn được sao chép từ software_safety-main-2 để chạy kiểm tra; không chỉnh sửa thư mục gốc. Các công cụ trong tools/ là phần bổ sung phục vụ kiểm chứng báo cáo. Dấu SHA-256 của tệp C được dùng:

```
8bebc57d194851ceaa0cc49adccad1591e7a46c7e9f7d15bd43f5e62411b53ee
```